

SistRUGBY-SLTC

Sistema de Gestión de Partidos y Estadísticas de Rugby
Santiago Lawn Tennis Club · Santiago del Estero

Trabajo Práctico N.º 4 (AP4)
Versión final integradora — Fase de Transición del PUD

Alumno: Fonzo, Ignacio
Licenciatura en Informática — Universidad Siglo 21
Seminario de Práctica — Módulo 4
Junio de 2026

Repositorio: <https://github.com/nachof9/SistRUGBY-SLTC>

1. Introducción

El presente documento corresponde al Trabajo Práctico N.º 4 (AP4) de la materia Seminario de Práctica de la Licenciatura en Informática de la Universidad Siglo 21. Constituye la entrega final integradora del proyecto SistRUGBY-SLTC y, en el marco del Proceso Unificado de Desarrollo (PUD), avanza con la fase de Transición: el prototipo deja de operar exclusivamente con datos en memoria y queda funcional con conexión real a una base de datos MySQL, tal como se solicitó en la retroalimentación de la entrega anterior.

Por su carácter iterativo e incremental, este trabajo retoma el análisis de negocio (AP1), el diseño y la base de datos (AP2) y el prototipo orientado a objetos (AP3), incorpora las correcciones señaladas por la cátedra y agrega las contribuciones específicas de AP4:

- Persistencia y consulta de datos en MySQL a través de JDBC: establecimiento de conexiones, consultas, actualización de registros y presentación de resultados en la interfaz.
- Selección y justificación del patrón de diseño coherente con la arquitectura, materializado además mediante una interfaz Java.
- Correcto manejo de excepciones en la interacción con MySQL.
- Inclusión de clases abstractas (Persona, EventoPartido) e interfaz (JugadorDAOInterface).
- Utilización complementaria de arreglos y de la clase ArrayList.
- Empleo de archivos para guardar y recuperar información (generación de reportes).

2. Resumen del proyecto

2.1 Problemática y justificación (modelo de negocio)

La sección de rugby del SLTC gestiona hoy a sus jugadores, partidos y estadísticas mediante planillas de cálculo individuales y registros en papel, sin integración. Esto provoca pérdida e inconsistencia de datos históricos, imposibilidad de generar reportes ágiles, dificultad para seguir el rendimiento individual y tiempo administrativo excesivo.

Desde la perspectiva del modelo de negocio, el cuerpo técnico y la secretaría deportiva son los clientes internos; su propuesta de valor es disponer de información oportuna y confiable para la toma de decisiones deportivas (convocatorias, planificación, seguimiento). Las actividades clave son el registro del padrón, la carga de partidos y eventos, y la generación de estadísticas; los recursos clave, una base de datos centralizada y una aplicación de escritorio sencilla. El proyecto se justifica porque elimina los costos ocultos del trabajo manual (retrabajo, errores, pérdida de datos) y habilita un activo reutilizable: el histórico deportivo del club.

2.2 Alcance

El sistema cubre las divisiones juveniles (M15 a M19) y el plantel superior (Pre-Intermedia, Intermedia y Primera). Quedan fuera del alcance las categorías infantiles, la gestión financiera, la transmisión en vivo y la integración con redes sociales. El producto es una aplicación de escritorio con persistencia en MySQL.

2.3 Síntesis de las entregas previas (carácter incremental)

- AP1 — Inicio (PUD): análisis del negocio, requerimientos funcionales (RF01–RF10) y no funcionales (RNF01–RNF07), procesos as-is, doce casos de uso y análisis de riesgos.
- AP2 — Elaboración (PUD): modelo de dominio, casos de uso detallados, diagramas de secuencia (CU01, CU05, CU07), arquitectura en tres capas, diagramas de clases / componentes / despliegue, modelo relacional normalizado a 3FN y DER.

- AP3 — Construcción (PUD): prototipo Java SE aplicando los cuatro pilares de POO, manejo de excepciones, estructuras de datos (lista enlazada, pila, cola, hashmap), algoritmos de ordenación (QuickSort) y búsqueda (binaria), y trazabilidad de CU01/CU02/CU07 hasta las pruebas.

Los diagramas UML y los documentos AP1–AP3 se conservan en la carpeta docs/ del repositorio.

3. Aplicación del Proceso Unificado de Desarrollo

El proyecto se desarrolló siguiendo las cuatro fases del PUD, garantizando calidad, escalabilidad y eficiencia mediante iteraciones con artefactos verificables:

Fase PUD	Entrega	Artefactos principales
Inicio	AP1	Visi&ocute;n, requerimientos, casos de uso, riesgos.
Elaboraci&ocute;n	AP2	Arquitectura 3 capas, modelo de dominio, DER, 3FN.
Construcci&ocute;n	AP3	Prototipo POO, estructuras, algoritmos, pruebas.
Transici&ocute;n	AP4	Conexi&ocute;n real a MySQL, reportes en archivo, producto desplegable.

La fase de Transición que materializa AP4 se concentra en convertir el prototipo en un producto desplegable: se completó la capa de persistencia contra MySQL, se incorporó la generación de reportes en archivos y se afinó el arranque del sistema para tolerar entornos con y sin base de datos.

4. Arquitectura y selección del patrón de diseño

4.1 Arquitectura en tres capas

SistRUGBY-SLTC adopta una arquitectura en capas (Layered Architecture): Presentación (MenuConsola recolecta entradas y muestra resultados), Negocio (clases *Service con la lógica de aplicación y las validaciones) y Persistencia (clases DAO que acceden a MySQL vía JDBC, con ConexionBD como Singleton). Cada capa conoce únicamente a la inmediatamente inferior, lo que produce bajo acoplamiento y alta cohesión.

4.2 Patrón de diseño seleccionado: DAO (Data Access Object)

La consigna solicita seleccionar y justificar el patrón de diseño coherente con la arquitectura. Se selecciona DAO (Data Access Object) como patrón protagonista de la solución.

Justificación. El DAO encapsula la totalidad del acceso a datos detrás de un contrato uniforme (insertar, findByDni, findByCategoria, findAll, darDeBaja), de modo que la capa de negocio nunca conoce detalles de JDBC ni de SQL. Esto resulta decisivo en este proyecto porque habilita el modo dual: la misma firma de método resuelve contra MySQL (rama JDBC) o contra un repositorio en memoria (rama demo) sin que los Service cambien una sola línea.

En esta entrega el contrato se materializa explícitamente mediante la interfaz Java JugadorDAOInterface, que JugadorDAO implementa. Así, la capa de negocio programa contra la abstracción y no contra la implementación (principio de inversión de dependencias), y se concreta —además de las clases abstractas Persona y EventoPartido— el uso de interfaces que la consigna admite como alternativa de abstracción.

```
public interface JugadorDAOInterface {
    Jugador insertar(Jugador j) throws SQLException;
    Jugador findByDni(String dni) throws SQLException;
    List<Jugador> findByCategoria(int idCategoria) throws SQLException;
    List<Jugador> findAll() throws SQLException;
    void darDeBaja(int idJugador) throws SQLException;
}

public class JugadorDAO implements JugadorDAOInterface { ... } // impl JDBC/memoria
```

```
public class JugadorService {
    private final JugadorDAOInterface dao = new JugadorDAO();
}
// capa de negocio
// depende de la interfaz
```

Todas las operaciones del contrato declaran `SQLException`, lo que centraliza en una única capa el manejo del error técnico de base de datos. Cualquier implementación futura del contrato (un DAO sobre un ORM u otro motor) podría sustituirse sin alterar la capa de negocio.

Criterio	Aporte del patrón DAO
Pros	Añade la persistencia; permite intercambiar el origen de datos (MySQL ↔ memoria) dentro de la misma aplicación.
Contras	Agrega una clase (e interfaz) por entidad; puede generar código repetitivo de mapeo fila ↔ objeto.
Riesgos	Si las firmas del contrato no se respetan, la abstracción se filtra; se mitiga con el método privado <code>mapa()</code> .

Patrones de soporte (Gamma et al., 1994): Singleton (`ConexionBD`: una única instancia de conexión compartida por todos los DAOs; `getInstance()` es `synchronized`); Factory Method (`EventoPartido.crear()`: decide qué subclase concreta de evento instanciar a partir del enum `Tipo`); y Layered Architecture como patrón arquitectónico macro.

5. Base de datos MySQL

5.1 Modelo relacional

El esquema se normalizó hasta la Tercera Forma Normal (3FN). Se compone de ocho tablas alineadas exactamente con las entidades del modelo Java: usuarios, categorías, clubes, temporadas, jugadores, partidos, `partido_plantel` y `eventos_partido`. Las relaciones se implementan con claves foráneas e integridad referencial (`ON UPDATE CASCADE / ON DELETE RESTRICT` o `CASCADE` según corresponda) sobre el motor InnoDB.

5.2 Fragmento del DDL

```
CREATE TABLE jugadores (
    id_jugador      INT      AUTO_INCREMENT PRIMARY KEY,
    nombre          VARCHAR(80) NOT NULL,
    apellido        VARCHAR(80) NOT NULL,
    dni             VARCHAR(10) NOT NULL UNIQUE,
    fecha_nacimiento DATE    NOT NULL,
    posicion        VARCHAR(30),
    id_categoria    INT      NOT NULL,
    estado          ENUM('activo','inactivo') NOT NULL DEFAULT 'activo',
    fecha_alta      DATE      NOT NULL DEFAULT (CURDATE()),
    fecha_baja      DATE,
    CONSTRAINT fk_jug_cat FOREIGN KEY (id_categoria)
        REFERENCES categorias (id_categoria)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE = InnoDB;
```

El script completo (`sql/schema.sql`) crea las ocho tablas con sus claves e índices; `sql/datos_iniciales.sql` carga los datos de prueba (ocho categorías, cinco clubes, tres temporadas, tres usuarios, cinco jugadores y un partido de muestra) e incluye consultas `SELECT` representativas.

6. Persistencia y consulta con JDBC

6.1 Conexión: Singleton y modo dual robusto

`ConexionBD` aplica Singleton y, en AP4, incorpora un modo dual robusto: si la variable de entorno `DB_PASS` está definida, intenta abrir la conexión JDBC real; si no lo está, o si la conexión falla (servidor apagado, credenciales incorrectas, driver ausente), el sistema cae automáticamente a un repositorio en memoria informando el motivo, en lugar de abortar.

```
private ConexionBD() {
    String url = System.getenv().getOrDefault("DB_URL", URL_DEFAULT);
    String user = System.getenv().getOrDefault("DB_USER", USER_DEFAULT);
    String pass = System.getenv().getOrDefault("DB_PASS", "");
    if (pass.isEmpty()) { this.modaMemoria = true; return; } // sin credenciales -> demo
    try { this.connection = DriverManager.getConnection(url, user, pass); }
    catch (SQLException e) { this.modaMemoria = true; } // fallback robusto, no aborta
}
```

6.2 Operaciones JDBC en los DAOs

Cada DAO resuelve la operación contra MySQL con PreparedStatement (consultas parametrizadas que previenen inyección SQL) y, en modo demo, contra RepositorioMemoria. Se ejemplifican las cuatro operaciones requeridas: establecer conexión, consultar, actualizar y presentar resultados.

```
// Alta (INSERT) con recuperacion de clave generada -- JugadorDAO
String sql = "INSERT INTO jugadores (nombre, apellido, dni, fecha_nacimiento, "
    + "posicion, id_categoria, estado, fecha_alta) "
    + "VALUES (?, ?, ?, ?, ?, ?, 'activo', CURDATE())";
try (PreparedStatement ps = bd.getConnection().prepareStatement(
    sql, Statement.RETURN_GENERATED_KEYS)) {
    ps.setString(1, j.getNombre()); ps.setString(2, j.getApellido());
    ps.setString(3, j.getDni()); ps.setDate(4, Date.valueOf(j.getFechaNacimiento()));
    ps.setString(5, j.getPosicion()); ps.setInt(6, j.getIdCategoria());
    ps.executeUpdate();
    try (ResultSet rs = ps.getGeneratedKeys()) { if (rs.next()) j.setId(rs.getInt(1)); }
}

// Consulta (SELECT) y mapeo fila->objeto -- JugadorDAO
String sql = "SELECT * FROM jugadores WHERE dni = ?";
try (PreparedStatement ps = bd.getConnection().prepareStatement(sql)) {
    ps.setString(1, dni);
    try (ResultSet rs = ps.executeQuery()) { if (rs.next()) return mapear(rs); }
}

// Actualizacion (UPDATE) -- baja logica que preserva el historial
String sql = "UPDATE jugadores SET estado='inactivo', fecha_baja=CURDATE() "
    + "WHERE id_jugador = ?";
```

6.3 Polimorfismo en la persistencia de eventos

EventoDAO guarda el discriminador tipo_evento = evento.getTipo().name() y, al leer, reconstruye la subclase concreta mediante la fábrica polimórfica EventoPartido.crear(...). Ni el DAO ni la base conocen las siete clases concretas.

```
private EventoPartido mapear(ResultSet rs) throws SQLException {
    EventoPartido.Tipo tipo = EventoPartido.Tipo.valueOf(rs.getString("tipo_evento"));
    EventoPartido e = EventoPartido.crear(tipo, rs.getInt("id_partido"),
        rs.getInt("id_jugador"), rs.getInt("minuto"));
    e.setId(rs.getInt("id_evento")); e.setDescripcion(rs.getString("descripcion"));
    return e;
}
```

7. Manejo de excepciones

El sistema distingue excepciones de dominio (reglas de negocio violadas) de excepciones técnicas (errores de E/S o de la base de datos). En la interacción con MySQL las operaciones declaran SQLException; la capa de presentación captura por especificidad (de la más concreta a la más genérica) y, ante el fallo de conexión, el modo dual evita la caída del sistema. La escritura de reportes maneja IOException de forma diferenciada.

Excepción	Tipo	Se lanza cuando
DniDuplicadoException	Dominio	Se intenta registrar un DNI ya existente.
JugadorNoEncontradoException	Dominio	Se referencia un id de jugador inexistente.

Excepciön	Tipo	Se lanza cuando…
CredencialesInvalidasException	Dominio	Usuario o contraseña no coinciden.
DatosInvalidosException	Dominio	Datos que violan reglas de validaciön.
SQLException	Técnic	Falla una operaciön JDBC contra MySQL.
IOException	Técnic	Falla la escritura del reporte en archivo.

8. Clases abstractas e interfaces. Pilares de POO

El prototipo materializa los cuatro pilares de la POO mediante dos jerarquías con clases abstractas y, en AP4, una interfaz de persistencia:

- Persona (abstracta) → Jugador, Usuario (herencia). Declara el contrato descripcionCorta().
- EventoPartido (abstracta) → Try, Conversion, Penal, Drop, TarjetaAmarilla, TarjetaRoja, Sustitucion. Declara calcularPuntos() y getTipo().
- JugadorDAOInterface (interfaz) → JugadorDAO. Define el contrato de persistencia (sección 4.2).

Abstracción y herencia se observan en esas jerarquías; el encapsulamiento, en los atributos private/protected con getters/setters y enums tipados; el polimorfismo, en calcularPuntos(), que devuelve el valor correcto según el tipo concreto sin recurrir a instanceof:

```
public int recalcularPuntosSLTC() {
    int total = 0;
    for (EventoPartido e : eventos) total += e.calcularPuntos(); // cada subclase responde
    return total;
}
```

9. Utilización complementaria de arreglos y ArrayList

La consigna exige emplear arreglos y la clase ArrayList de forma complementaria. El sistema usa cada estructura donde es óptima:

- ArrayList (tamaño dinámico) para el ranking de jugadores, cuya cantidad de elementos se desconoce de antemano y crece según los datos.
- Arreglo int[] (tamaño fijo) para el resumen de eventos por tipo: su tamaño es conocido en compilación (la cantidad de valores del enum Tipo) y el acceso es O(1) indexando por ordinal(), sin boxing.

```
public int[] conteoGlobalPorTipo() throws SQLException {
    int[] conteo = new int[EventoPartido.Tipo.values().length]; // ARREGLO fijo
    for (Partido p : partidoDao.findAll())
        for (EventoPartido e : eventoDao.findByPartido(p.getId()))
            conteo[e.getTipo().ordinal()]++; // acceso O(1)
    return conteo;
}
// El ranking, en cambio, se devuelve como ArrayList<FilaRanking> (tamano dinamico).
```

10. Empleo de archivos

ReporteService (CU09) exporta el ranking y el resumen de eventos a un archivo de texto. La escritura usa try-with-resources sobre un BufferedWriter (cierra automático del recurso) y maneja IOException, combinando el ArrayList del ranking con el arreglo int[] del resumen:

```
try (BufferedWriter w = Files.newBufferedWriter(destino, StandardCharsets.UTF_8)) {
    w.write("RANKING DE JUGADORES POR PUNTOS"); w.newLine();
    for (FilaRanking f : ranking) { /* ... */ w.newLine(); }
    EventoPartido.Tipo[] tipos = EventoPartido.Tipo.values(); // arreglo
    for (int i = 0; i < tipos.length; i++)
        w.write(String.format("%-18s : %d", tipos[i], conteoPorTipo[i]));
}
```

```
}
```

11. Estructuras de datos y algoritmos

Se conservan, del prototipo AP3, las estructuras y algoritmos con justificación funcional:

Estructura / algoritmo	Uso justificado	Complejidad
LinkedList<EventoPartido>;	Eventos cronológicos de un partido (append O(1) inserción	O(1)
ArrayDeque (pila LIFO)	Deshacer el último evento cargado (undo).	O(1) push/pop
Queue (FIFO)	Orden de sustituciones planificadas.	O(1) offer/poll
HashMap	Acceso por id en modo memoria.	O(1) amortizado
QuickSort (mediana de tres)	Orden alfabético de jugadores.	O(n log n)
Búsqueda binaria	Localización por DNI sobre lista ordenada.	O(log n)

12. Plan de pruebas y evidencia

12.1 Estrategia

Se combinaron pruebas funcionales de caja negra, pruebas de integración (comunicación entre capas), pruebas de seguridad (verificación de hash) y pruebas de regresión. El prototipo se ejecuta de forma scripteada alimentando el menú con un archivo de entrada (smoke_input.txt).

12.2 Resultados

ID	Caso / objeto	Tipo	Criterio de éxito	Result.
PT01	CU01 Iniciar sesión	Funcional	Acceso con rol correcto	APROBADO
PT02	CU01 (negativo)	Seguridad	Bloqueo tras 3 intentos	APROBADO
PT03	CU02 Registrar jugador	Integración	Jugador persistido con id auto	APROBADO
PT04	CU02 (DNI duplicado)	Negocio	Rechazo con DniDuplicadoException	APROBADO
PT05	CU04 Dar de baja	Funcional	estado=inactive; historial intacto	APROBADO
PT06	CU05 Registrar partido	Integración	Partido persistido; id generado	APROBADO
PT07	CU06 Registrar plantel	Funcional	Plantel con titular obligatorio	APROBADO
PT08	CU07 Registrar evento	Polimorfismo	Evento + registro de marcador	APROBADO
PT09	CU08 Estadísticas	Algoritmo	Ranking ordenado por puntos	APROBADO
PT10	Pila undo	Estructuras	Pop LIFO + registro automático	APROBADO
PT11	Búsqueda binaria	Algoritmo	Localiza por DNI en O(log n)	APROBADO
PT12	QuickSort	Algoritmo	Lista ordenada por apellido	APROBADO
PT13	Conexión MySQL real	Integración	INSERT/SELECT/UPDATE persistidos	APROBADO

12.3 Evidencia de ejecución (modo demo, sin MySQL)

```
[INFO] DB_PASS no definida (modo demo en memoria).
[INFO] Modo demo (en memoria). Cargando datos de prueba...
--- Ranking por puntos (POLIMORFISMO en calcularPuntos) ---
JUGADOR      | TRY | CONV | PEN | DROP | PUNTOS
Casas, Federico | 1   | 1   | 1   | 0   | 10
Pereyra, Rodrigo | 1   | 0   | 0   | 0   | 5
```

12.4 Evidencia de ejecución con conexión real a MySQL

Atendiendo de manera directa la observación de la cátedra —«en la cuarta entrega debe ya estar con conexión a la base... algo funcional y con conexión a la base de datos»—, se ejecutó el sistema contra una instancia real de MySQL 8.0 y se verificó el ciclo completo de persistencia. El controlador

mysql-connector-j 8.3.0 se carga en el classpath y, al definir DB_PASS, el arranque ya no muestra el mensaje de modo demo: la aplicación abre la conexión JDBC y opera directamente sobre la base sistrugby_sltc.

```

Consola - App conectada a MySQL (INSERT + SELECT + polimorfismo)

PS C:\...\SistRUGBY-SLTC> $env:DB_PASS='sltc2026'
PS C:\...\SistRUGBY-SLTC> java -cp 'out;lib/mysql-connector-j-8.3.0.jar' com.sltc.sistrugby.Main

[INFO] Conectado a MySQL (modo produccion). Los datos provienen de la base sistrugby_sltc.
=====
SistRUGBY-SLTC | Santiago Lawn Tennis Club | Prototipo AP4
=====

--- Iniciar sesion ---
Usuario: entrenador1
Contraseña: *****
Bienvenido, entrenador1 [ENTRENADOR]

--- CU02 Registrar jugador ---
Nombre: Pedro Apellido: Gomez DNI: 40123456
Fecha nac. (YYYY-MM-DD): 2000-03-15 Posicion: Pilar Categoria: 8
OK. Jugador creado: Jugador #6 - Gomez, Pedro [Pilar] - DNI 40123456

--- Ranking por puntos (POLIMORFISMO en calcularPuntos) ---
JUGADOR | TRY | CONV | PEN | DROP | PUNTOS
-----|-----|-----|-----|-----|-----
Casas, Federico | 1 | 1 | 1 | 0 | 10
Pereyra, Rodrigo | 1 | 0 | 0 | 0 | 5

```

Figura 17 — Arranque conectado a MySQL. Se realiza el alta del jugador #6 (INSERT, CU02) y se consulta el ranking polimórfico (SELECT, CU08) sobre datos de la base.

La persistencia es real: el jugador insertado por la aplicación se recupera con una consulta independiente desde el cliente de MySQL, demostrando que la fila quedó almacenada en la base y no en memoria.

```

MySQL CLI - La fila insertada por la app esta en la base

mysql> USE sistrugby_sltc;
mysql> SELECT id_jugador, apellido, nombre, dni, estado
-> FROM jugadores ORDER BY id_jugador;
+-----+-----+-----+-----+-----+
| id_jugador | apellido | nombre | dni | estado |
+-----+-----+-----+-----+-----+
| 1 | Pereyra | Rodrigo | 35211001 | activo |
| 2 | Villalba | Matias | 36089452 | activo |
| 3 | Casas | Federico | 34567890 | activo |
| 4 | Herrera | Gonzalo | 37124500 | activo |
| 5 | Ruiz | Tomas | 35980012 | activo |
| 6 | Gomez | Pedro | 40123456 | activo | <-- alta persistida en MySQL
+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

```

Figura 18 — Verificación directa en MySQL: la fila insertada por la app (jugador #6, Gomez) está persistida en la tabla jugadores.

Finalmente, ReporteService (CU09) exporta el ranking (ArrayList) y el resumen por tipo de evento (arreglo int[]) a un archivo de texto, complementando la persistencia en MySQL con un soporte de salida apto para imprimir o compartir:


```
Persistencia en ARCHIVO: ArrayList (ranking) + arreglo int[] (resumen)

$ type reporte_sltc.txt  (archivo generado por ReporteService - CU09)

=====
SistRUGBY-SLTC - Reporte de estadísticas (CU09)
Generado: 2026-06-28 20:51:51
=====

RANKING DE JUGADORES POR PUNTOS
JUGADOR                | TRY | CONV | PEN | DROP | PUNTOS
-----
Casas, Federico        | 1   | 1    | 1   | 0    | 10
Pereyra, Rodrigo       | 1   | 0    | 0   | 0    | 5

RESUMEN GLOBAL POR TIPO DE EVENTO
-----
TRY                : 2
CONVERSION         : 1
PENAL              : 1
DROP               : 0
TARJETA_AMARILLA   : 0
TARJETA_ROJA       : 0
SUSTITUCION        : 0

Fin del reporte.
```

Figura 19 — Archivo `reporte_sltc.txt` generado por `ReporteService`, combinando `ArrayList` (ranking) y arreglo `int[]` (resumen).

Se validó además la robustez del modo dual (PT13): con el servidor MySQL detenido, el sistema informa el motivo y conmuta automáticamente al repositorio en memoria en lugar de abortar. La conexión real y el fallback quedan así demostrados como dos caminos verificados del mismo arranque, lo que facilita tanto la operación en el club como la corrección académica.

13. Atención de las observaciones previas

- AP3 (docente): «en la cuarta entrega debe ya estar con conexión a la base... algo funcional y con conexión a la base de datos.» → AP4 completa el JDBC de todos los DAOs (eventos, partidos, plantel, categorías, clubes y temporadas), unifica `schema.sql` con el código, entrega hashes PBKDF2 reales para el login en MySQL y aporta evidencia de ejecución conectada (sección 12.4).
- Patrón e interfaces: se explicitó el contrato del patrón DAO mediante la interfaz `JugadorDAOInterface`, de modo que la afirmación de diseño se verifica en el código.
- POO: se ratifican encapsulamiento, herencia, polimorfismo y abstracción, con revisión de constructores (por defecto, de alta y completo) y de la creación de objetos.
- AP2: la portada no contiene encabezado ni pie de página y se mantiene la trazabilidad CU → código → pruebas.

14. Repositorio y guía de ejecución

Repositorio público: <https://github.com/nachof9/SistRUGBY-SLTC>

Modo demo (sin MySQL):

```
javac -d out -encoding UTF-8 $(find src/main/java -name "*.java")
java -cp out com.sltc.sistrugby.Main
```

Modo producción (con MySQL):

```
mysql -u root -p < sql/schema.sql
mysql -u root -p < sql/datos_iniciales.sql
set DB_PASS=tu_password
java -cp "out;lib/mysql-connector-j-8.3.0.jar" com.sltc.sistrugby.Main
```

Credenciales de prueba: admin/admin2026, entrenador1/rugby2026, secretario1/sltc2026.

15. Conclusiones

AP4 cierra el ciclo del PUD para SistRUGBY-SLTC entregando un producto funcional y con conexión real a MySQL, verificada mediante alta, consulta y actualización de registros sobre la base. Se completó la capa de persistencia en todos los DAOs, se seleccionó y justificó el patrón DAO como eje de la solución —materializado en una interfaz Java y apoyado en Singleton y Factory Method—, se incorporaron arreglos y ArrayList de manera complementaria, y se agregó la generación de reportes en archivos. El modo dual robusto permite ejecutar el sistema con o sin base de datos. La arquitectura en capas y la separación que impone el patrón DAO dejan el sistema preparado para futuras extensiones (interfaz gráfica Swing, exportación a PDF/CSV, pruebas unitarias con JUnit) sin reescribir la lógica existente.

16. Referencias

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.

Jacobson, I., Booch, G., & Rumbaugh, J. (1999). The Unified Software Development Process. Addison-Wesley.

Kendall, K., & Kendall, J. (2011). Análisis y diseño de sistemas (8.ª ed.). Pearson Educación.

Larman, C. (2003). UML y patrones (2.ª ed.). Pearson Prentice Hall.

National Institute of Standards and Technology. (2010). NIST SP 800-132. NIST.

Oracle Corporation. (2024). Java SE 17 Documentation. <https://docs.oracle.com/en/java/javase/17/>

MySQL AB. (2024). MySQL 8.0 Reference Manual. <https://dev.mysql.com/doc/refman/8.0/en/>